

#Homio - Rental Property Management App

****Module:**** OPSC6322 - Open Source Coding

****Croup Members****

- Letlhogonolo Chabalala - ST10333694

- Fortunate Lukhele - ST10443192

- Palesa Mhlana - ST10459586

1. Purpose of the App

Tenants and Landlords in South Africa juggle WhatsApp, email, paper lease documents and informal payment records to manage a rental - which makes information hard to track. Homio is a mobile app that brings rent tracking, maintenance requests, lease documents, property inspections, and tenant-landlord communication into a single place. The app is built using Kotlin and connects to Firebase for the backend (authentication + database).

What the app does

There's two types of users - Tenants and Landlords. When you register you pick which one you are and the app shows you different screens depending on that.

****Landlords can:****

- Add and manage their properties
- Assign tenants to a property (by their email)
- See a list of their tenants

- View and respond to maintenance requests
- Schedule inspections and mark them as completed
- View proof of payment that tenants upload
- Message their tenants

****Tenants can:****

- View their rental info (rent amount, deposit, property details)
- Submit maintenance requests
- View their inspections
- Upload proof of payment
- Message their landlord
- Edit their profile

There's also a Settings page where you can change the app language (English, Afrikaans, isiZulu and isiXhosa - still working on getting all the screens translated, right now its mostly just the settings screen itself) and switch between light and dark mode.

Tech stack

- Kotlin
- Firebase Authentication (for login/register)
- Firebase Firestore (database)
- RecyclerView for all the lists
- Glide for loading images
- CI/CD (GitHub Actions)

Screenshots

Login Screen

3:36



Email

Password

LOGIN

Don't have an account? Sign Up

Register Screen

3:37



Full Name

Email

Password

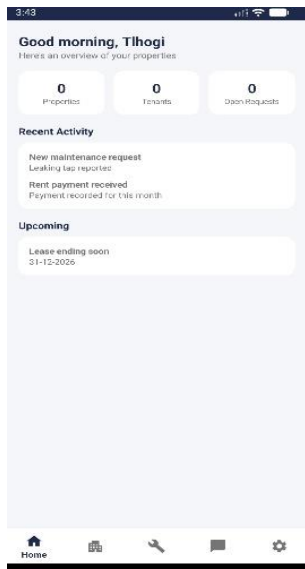
Confirm Password

I am: ☒ Tenant ☐ Landlord

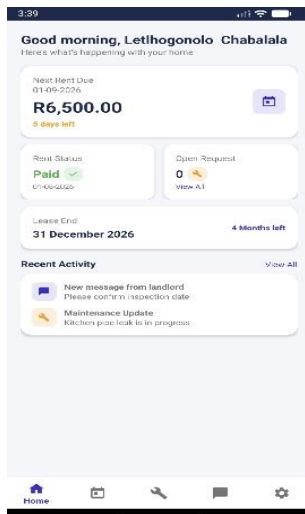
SIGN UP

Already have an account? Login

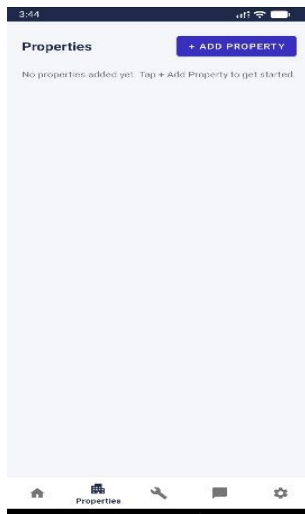
Landlord Dashboard



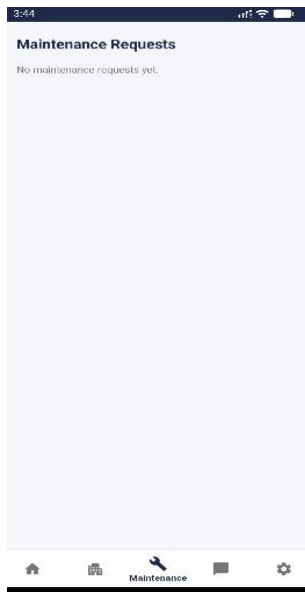
Tenant Dashboard



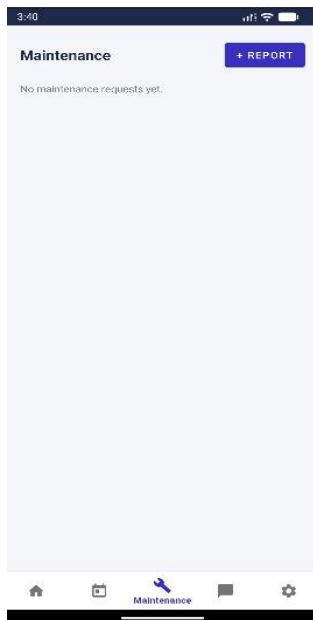
Add Property



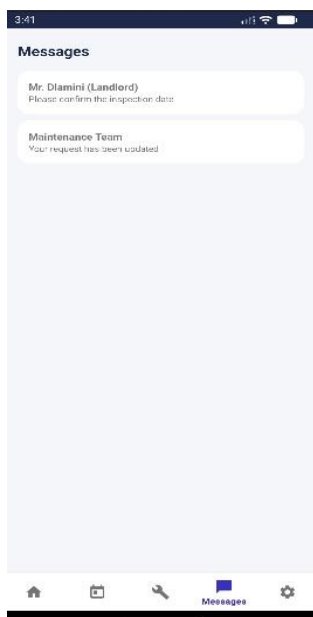
Maintenance Requests (Landlord)



Submit Maintenance Request (Tenant)



Messages



How the app is structured

The project follows a pretty basic structure:

- `models/` - data classes like Property, Maintenance, Payment, Inspection, Message

- `adapters/` - RecyclerView adapters for the lists
- `utils/` - helper classes (SettingsPrefs for theme, etc)
- Activities are all in the main package

Firestore has these collections: `users`, `properties`, `maintenance`, `payments`, `inspections`, `messages`

Unit tests

Tests cover `PasswordUtil` (hashingcorrectness), `Validators` (email/password/required-field validation)

Known issues / things I still need to fix

- No profile picture upload yet
- The "Recent Activity" section on the landlord dashboard doesn't actually show real activity yet, its just an empty state for now
- Need to double check the login bug where it sometimes fails right after registering
- Firebase Authentication require an internet connection for cloud-based authentication
- Push notifications, digital inspections photo capture, full runtime multi-language translations and offline synchronization will be completed during the final POE

Setup (for anyone testing this)

1. Clone the repo
2. Open in Android Studio
3. Add your own `google-services.json` file in the `app/` folder (I didn't include mine for obvious reasons)

4. Make sure Email/Password sign in is turned on in Firebase console

5. You'll need to set up your own Cloudinary account for the image uploads to work - just needs a cloud name and an unsigned upload preset

4. App Architecture & Structure

The application follows a clean, modular package architecture:

```
``text
```

```
com.homio.app/
```

```
|--adapters/ # RecyclerView adapters for lists (Properties, Maintenance, Payments, Messages)
```

```
|-- models/ # Kotlin data classes (Property, Maintenance, Payment, Inspection, Message, User)
```

```
|--utils/ # Helper modules (SettingsPrefs for themes/language, CloudinaryHelper, Constants)
```

```
|-- views/ # Activity and Fragment UI controllers
```